

AuthKit Plugin -- Integration & Developer Guide

Universal PlayFab Authentication Plugin for Google, Facebook & Instagram in Unity

UNITY PACKAGE (UPM / .UNITYPACKAGE) - VERSION 1.0.0

1. Overview & Architecture

AuthKit is a modular, zero-coupling authentication package designed to be dropped into **any Unity game project** (e.g. Sudoku, RPG, arcade). It bridges Google Account and Meta (Facebook / Instagram) logins to **Microsoft PlayFab** backend seamlessly.



Zero Cross-Project Coupling: Credentials and PlayFab settings live inside a dedicated **AuthSettings.asset** ScriptableObject per project. No hardcoded keys, project IDs, or package names exist inside the plugin code.

2. Prerequisites in the Consuming Project

Prerequisite	Purpose	Installation
PlayFab Unity SDK	Cloud player account management and data storage.	Import PlayFabSDK.unitypackage from PlayFab Developer Portal.
Facebook Unity SDK	Native Meta authentication for Facebook and Instagram.	Import facebook-sdk.unitypackage from Meta Developers Portal.
EDM4U (External Dependency Manager)	Auto-resolves Google Play Services auth dependencies on Android.	Bundled with PlayFab / Google plugins or downloaded as unitypackage.

3. Step-by-Step Installation into Any Project

- Export from AuthKit:** In the AuthKit Unity project, click **Auth Login -> Export .unitypackage**. The file **build/AuthLogin_v1.0.0.unitypackage** is generated.
- Import into Target (e.g. Sudoku):** In your target project, click **Assets -> Import Package -> Custom Package...**, select the package and click **Import**.
- Create Settings Asset:** Navigate to **Assets/Resources/**, right-click -> **Create -> Auth Login -> Auth Settings** and name it **AuthSettings**.

AuthSettings Configuration Fields:

Field	Required For	Description
PlayFab Title ID	All (Optional)	Your PlayFab Title ID (e.g. AB12C). If left blank, falls back to PlayFab SDK Title ID.
Google Web Client ID	Google Login	OAuth 2.0 Web application Client ID from Google Cloud Console.
Facebook App ID	Facebook / Instagram	App ID from Meta Developer Dashboard (developers.facebook.com).
Facebook Client Token	Facebook / Instagram	Client Token from Meta Dashboard -> App Settings -> Advanced.

4. Platform Configurations Checklist

A. PlayFab Game Manager (developer.playfab.com)

- Navigate to **Title Settings** -> **Add-ons**.
- **Google:** Enable and paste your OAuth **Web application Client ID** and **Client Secret**.
- **Facebook:** Enable and paste your **Facebook App ID** and **App Secret**.

B. Google Cloud Console (console.cloud.google.com)

- **Web Application Client:** Create an OAuth Client ID of type *Web application*. Copy this ID into `AuthSettings.asset` and PlayFab Google Add-on.
- **Android Client:** Create an OAuth Client ID of type *Android*. Enter your game's Android Package Name (e.g. `com.yourname.sudoku`) and your Keystore SHA-1 fingerprint.

C. Meta for Developers (developers.facebook.com)

- Create an App with the **Facebook Login** product enabled.
- Under **App Settings** -> **Basic** -> **Android**: Add your Package Name and Key Hash (Base64 SHA-1).
- Copy App ID and Client Token into your `AuthSettings.asset`.

5. Code Integration & API Reference

Single-Line Sign In Triggers

Wire these directly in the Unity Inspector to UI Button `OnClick()` events:

```
// 1. Google Account Sign In
AuthManager.Instance.SignInWithGoogle();

// 2. Facebook Sign In
AuthManager.Instance.SignInWithFacebook();

// 3. Instagram Sign In (Meta login with Instagram scopes)
AuthManager.Instance.SignInWithInstagram();

// 4. Sign Out
AuthManager.Instance.SignOut();
```

Listening to Authentication Events

```
using UnityEngine;
using SadibTools.AuthLogin;

public class LoginUIController : MonoBehaviour
{
    private void OnEnable()
    {
        var auth = AuthManager.EnsureInstance();
        auth.OnLoginStarted += HandleStarted;
        auth.OnLoginSuccess += HandleSuccess;
        auth.OnLoginFailure += HandleFailure;
    }

    private void OnDisable()
    {
        if (AuthManager.Instance == null) return;
        AuthManager.Instance.OnLoginStarted -= HandleStarted;
        AuthManager.Instance.OnLoginSuccess -= HandleSuccess;
        AuthManager.Instance.OnLoginFailure -= HandleFailure;
    }

    private void HandleStarted(string providerId) => Debug.Log($"Connecting to {providerId}...");

    private void HandleSuccess(AuthSession session)
    {
        Debug.Log($"Logged in via {session.ProviderId}! PlayFab ID: {session.PlayFabId}, Name: {session.DisplayName}");
        // Proceed to main game / load player cloud data
    }

    private void HandleFailure(AuthError error)
    {
        Debug.LogError($"Login failed: [{error.Code}] {error.Message}");
    }
}
```

6. Troubleshooting & Common Pitfalls

Symptom	Root Cause	Solution
Google error 10 / 12500	SHA-1 fingerprint mismatch or wrong Package Name.	Ensure the Android OAuth Client in Google Cloud Console has the exact SHA-1 of the keystore used to sign the APK.

Facebook Invalid Key Hash	Meta portal missing the APK key hash.	Generate the base64 SHA-1 key hash from your debug/release keystore and paste it into Meta App Settings.
Google Login fails in Editor	Google Identity Authorization API requires Android OS.	Google login is protected by <code>#if UNITY_ANDROID &&!UNITY_EDITOR</code> . Build and test on a real Android device.